

UNIVERSITÉ DU SINE SALOUM EL HADJI IBRAHIMA NIASS

UFR des Sciences Fondamentales et de l'Ingénierie
Département de Mathématiques-Informatique

Travaux Dirigés Algorithmique sur Arduino

Thème : Tableaux et Contraintes Embarquées

Enseignant : Dr B. DIOP

Objectif : Réadapter les problèmes algorithmiques classiques aux contraintes d'un microcontrôleur (2 Ko de RAM, 32 Ko de Flash, 16 MHz).

Partie I : Parcours et Recherche sous Contraintes Mémoire

Exercice 1. Recherche des extremums sur flux de capteurs

Un capteur de température envoie des mesures en continu sur la broche analogique A0. On souhaite déterminer, **en un seul parcours et sans stocker toutes les valeurs**, la température maximale et la deuxième plus grande température sur les 1000 dernières lectures.

Contrainte Arduino : Stocker 1000 entiers nécessiterait 2000 octets, soit la totalité de la SRAM !

Travail demandé :

1. Écrire un programme qui lit 1000 valeurs et trouve les deux plus grandes **sans tableau**
2. Afficher les résultats avec `Serial.println(F(...))` pour économiser la RAM
3. Mesurer la RAM libre avec `freeRam()` pour prouver l'efficacité

Prototype suggéré :

```
1 void trouverDeuxMax(int* max1, int* max2, int nbLectures);
```

Question algorithmique : Pourquoi la contrainte $O(n)$ en un seul parcours devient-elle une nécessité absolue ici, alors qu'elle n'est qu'une optimisation sur PC ?

Exercice 2. Vérification de tendance monotone

Un système de surveillance vérifie si la pression dans une cuve augmente de manière continue (risque d'explosion).

Travail demandé :

1. Écrire une fonction qui vérifie si les N dernières lectures sont strictement croissantes
2. Version A : avec un tableau de taille N (calculer la RAM consommée)

3. Version B : **sans tableau**, en ne gardant que la valeur précédente

```
1 // Version A - Gourmande en RAM
2 bool estCroissant_v1(int tab[], int taille);
3
4 // Version B - Optimisee Arduino
5 bool estCroissant_v2(int nouvelleLecture); // A appeler a chaque
   lecture
```

Question : Pour $N = 500$ lectures, combien d'octets économise la version B ?

Exercice 3. Comparaison de séquences de capteurs

Deux capteurs de distance (ultrasons) mesurent simultanément. On veut détecter les moments où ils donnent la même lecture (objet centré entre les deux).

Travail demandé :

1. Stocker 50 mesures de chaque capteur
2. Compter combien de fois les deux capteurs ont donné la même valeur au même instant
3. **Optimisation :** Peut-on faire cette comparaison *sans* stocker les 100 valeurs ?

```
1 int comparerCapteurs(const int t1[], const int t2[], int taille);
```

Extension Arduino : Utiliser une interruption timer pour déclencher les lectures à intervalles réguliers.

Exercice 4. Détection de pattern lumineux

Un récepteur infrarouge capture une séquence de 32 bits représentant un code télécommande. On veut vérifier si le code reçu correspond à un code attendu.

Travail demandé :

1. Stocker les codes valides en **PROGMEM** (économie de RAM)
2. Écrire une fonction qui compare un code reçu avec les codes en Flash

```
1 // Codes stockes en Flash
2 const uint32_t codesValides[] PROGMEM = {0xFFA25D, 0xFF629D, 0xFFE21D};
3
4 bool codeEstValide(uint32_t codeRecu);
```

Question : Pourquoi ne peut-on pas utiliser `==` directement avec un tableau **PROGMEM** ?

Exercice 5. Vérification de sous-ensemble de configurations

Un système domotique gère 16 actionneurs (relais). L'utilisateur envoie une liste de relais à activer. On doit vérifier que tous les relais demandés existent dans la configuration matérielle.

Travail demandé :

1. Représenter les 16 relais disponibles par un `uint16_t` (1 bit = 1 relais)
2. Représenter la demande utilisateur de la même façon
3. Vérifier que la demande est un "sous-ensemble" des relais disponibles avec une opération bit à bit

```
1 bool estSousEnsemble(uint16_t demande, uint16_t disponible);
2 // Indice : utiliser l'opération AND
```

Comparaison : Sur PC, on utiliserait deux tableaux et des boucles imbriquées $O(n \times m)$. Ici, une seule instruction suffit !

Exercice 6. Détection de séquence continue

Un encodeur rotatif sur les broches 2 et 3 génère des impulsions. On veut vérifier que les positions lues sont bien consécutives (pas de “saut” indiquant un dysfonctionnement).

Travail demandé :

1. Utiliser les interruptions pour capturer chaque changement de position
2. Stocker les 20 dernières positions dans un **buffer circulaire** (économie de mémoire)
3. Vérifier si les positions sont successives

```
1 volatile int buffer[20];
2 volatile byte indexEcriture = 0;
3
4 bool positionsSuccessives();
```

Question algorithmique : Qu'est-ce qu'un buffer circulaire et pourquoi est-il adapté aux systèmes embarqués ?

Exercice 7. Recherche dichotomique en Flash

Une table de calibration de 500 entrées associe des valeurs ADC (0-1023) à des températures réelles. La table est triée par valeur ADC.

Contrainte : 500×4 octets = 2000 octets, impossible en SRAM !

Travail demandé :

1. Stocker la table en PROGMEM
2. Implémenter une recherche dichotomique adaptée (utiliser `pgm_read_word`)
3. Retourner la température correspondant à une lecture ADC

```
1 // Structure en Flash
2 struct Calibration {
3     uint16_t adc;
4     int16_t temperature; // en dixiemes de degres
5 };
6
7 const Calibration table[500] PROGMEM = { ... };
8
9 int16_t chercherTemperature(uint16_t valeurADC);
```

Question : La complexité $O(\log n)$ est-elle plus importante sur Arduino que sur PC ? Justifier.

Partie II : Statistiques et Comptage avec Mémoire Limitée

Exercice 8. Filtrage des valeurs aberrantes

Un capteur de luminosité peut donner des lectures parasites. On veut identifier les valeurs qui n'apparaissent qu'une seule fois (probables erreurs) parmi les 100 dernières lectures.

Contrainte : On ne peut pas créer un tableau de comptage de taille 1024 (toutes les valeurs ADC possibles).

Travail demandé :

1. Version naïve : tableau de 100 valeurs + double boucle $O(n^2)$
2. Version optimisée : utiliser seulement 8 octets pour les valeurs 0-63 (avec des bits)
3. Comparer la consommation mémoire

Question : Pourquoi l'algorithme $O(n^2)$ devient-il acceptable quand n est petit (100) mais que la mémoire est le facteur limitant ?

Exercice 9. Histogramme de mesures

On mesure la luminosité ambiante et on veut créer un histogramme sur 10 plages (0-99, 100-199, ..., 900-1023).

Travail demandé :

1. Créer un tableau de 10 compteurs (byte suffit si < 256 occurrences)
2. Classifier 200 lectures dans les bonnes plages
3. Afficher l'histogramme sur le port série

```
1 byte histogramme[10]; // Seulement 10 octets !
2
3 void ajouterMesure(int valeur);
4 void afficherHistogramme();
```

Extension : Sauvegarder l'histogramme en EEPROM pour le conserver après coupure de courant.

Exercice 10. Détection d'état majoritaire

Un bouton est lu 100 fois en 10 ms pour filtrer les rebonds. On cherche l'état majoritaire (HIGH ou LOW).

Travail demandé :

1. Implémenter l'algorithme de Boyer-Moore (vote majoritaire) qui utilise $O(1)$ mémoire
2. Comparer avec l'approche naïve (compteur de HIGH et LOW)

```
1 // Boyer-Moore : O(n) temps, O(1) espace
2 int etatMajoritaire(byte broche, int nbLectures);
```

Question : Cet algorithme est-il plus avantageux sur Arduino que sur PC ? Pourquoi ?

Exercice 11. Bilan énergétique

Un système mesure la production solaire (valeurs positives) et la consommation (valeurs négatives) chaque seconde.

Travail demandé :

1. Sans tableau : calculer en temps réel la somme des productions et des consommations
2. Afficher le bilan (différence) toutes les 60 secondes
3. Sauvegarder le bilan journalier (1440 valeurs) en EEPROM (compression nécessaire !)

Défi : Comment stocker 1440 valeurs (journée) dans 1 Ko d'EEPROM ?

Exercice 12. Premier identifiant libre

Un système gère jusqu'à 32 périphériques I2C, chacun ayant un ID de 0 à 31. Quand un nouveau périphérique se connecte, on doit lui attribuer le plus petit ID libre.

Travail demandé :

1. Représenter les IDs utilisés par un `uint32_t` (1 bit par ID)
2. Trouver le premier bit à 0 (premier ID libre)

[illegible]

Comparaison : Sur PC, on utiliserait un tableau de booléens (32 octets). Ici, 4 octets suffisent !

Partie III : Modifications et Buffers Circulaires

Exercice 13. Suppression des doublons temps réel

Un capteur RFID lit des badges. On ne veut pas traiter deux fois le même badge dans un intervalle de 5 secondes.

Travail demandé :

1. Maintenir un buffer des 10 derniers badges lus (IDs sur 4 octets)
2. À chaque lecture, vérifier si le badge est déjà dans le buffer
3. Si nouveau, l'ajouter et déclencher l'action ; sinon, ignorer

```
1 uint32_t  derniersIDs[10];
2 unsigned long timestamps[10];
3
4 bool  nouveauBadge(uint32_t id);
```

Question : Pourquoi utiliser un buffer de taille fixe plutôt qu'une liste dynamique ?

Exercice 14. Buffer circulaire d'échantillons audio

Un micro échantillonne le son à 8 kHz. On doit maintenir un buffer des 256 derniers échantillons pour une FFT.

Travail demandé :

1. Implémenter un buffer circulaire de 256 octets
2. Utiliser une interruption timer pour l'échantillonnage
3. Gérer l'insertion avec manipulation d'index circulaire

```
1 volatile byte buffer[256];
2 volatile byte indexEcriture = 0;
3
4 // ISR appelee toutes les 125 us (8 kHz)
5 ISR(TIMER1_COMPA_vect) {
6     buffer[indexEcriture] = analogRead(A0) >> 2; // 10 bits -> 8 bits
7     indexEcriture = (indexEcriture + 1) & 0xFF; // Modulo 256 rapide
8 }
```

Question : Pourquoi `& 0xFF` est-il plus rapide que `% 256` ?

Exercice 15. Insertion dans table de calibration triée

On peut ajouter des points de calibration à la table de l'exercice 7 pendant l'exécution (mode apprentissage). La table en RAM (limitée à 20 entrées) doit rester triée.

Travail demandé :

1. Écrire `insérerCalibration(uint16_t adc, int16_t temp)` avec décalage des éléments
2. Fusionner la table RAM avec la table PROGMEM lors des recherches

Question : Pourquoi le décalage $O(n)$ est-il acceptable pour $n = 20$ mais serait problématique pour $n = 1000$?

Exercice 16. File d'attente de commandes

Un système reçoit des commandes série et les exécute dans l'ordre. Si une commande est annulée, elle doit être retirée de la file.

Travail demandé :

1. Implémenter une file (FIFO) de 8 commandes maximum
2. Écrire `enfiler(byte cmd)`, `defiler()`, et `supprimerCommande(byte cmd)`
3. Gérer la suppression avec compactage du tableau

```
1 byte fileCommandes[8];
2 byte taille = 0;
```

Exercice 17. Compactage de données capteur

Un capteur peut renvoyer 0 pour indiquer “pas de mesure valide”. Avant de transmettre un paquet de 16 mesures par radio, on veut compacter les données en déplaçant les zéros à la fin.

Travail demandé :

1. Implémenter le compactage en place (sans tableau auxiliaire)
2. Retourner le nombre de mesures valides
3. Transmettre uniquement les mesures valides pour économiser la bande passante

```
1 int compacterMesures(int mesures[], int taille);
```

Exercice 18. Affichage binaire sur LEDs

8 LEDs sont connectées sur le PORTD. Afficher la valeur d'un capteur (0-255) en binaire.

Travail demandé :

1. Lire une valeur analogique et la convertir en 8 bits
2. Afficher directement sur les 8 LEDs avec écriture dans PORTD
3. Comparer avec une version utilisant 8 appels à `digitalWrite()`

```
1 void afficherBinaire(byte valeur) {  
2     PORTD = valeur; // Une seule instruction !  
3 }
```

Mesure : Chronométrer les deux versions avec `micros()`.

Partie IV : Tri et Fusion pour Données Temps Réel**Exercice 19. Tri pour médiane glissante**

Pour filtrer le bruit d'un capteur, on calcule la médiane des 5 dernières lectures (au lieu de la moyenne, plus robuste aux valeurs aberrantes).

Travail demandé :

1. Maintenir un buffer de 5 valeurs
2. À chaque nouvelle lecture, trier le buffer et retourner la médiane
3. Optimiser : ne pas trier tout le tableau (tri partiel ou réseau de tri)

```
1 int medianeGlissante(int nouvelleLecture);
```

Question : Pourquoi un tri à bulles optimisé (qui s'arrête si déjà trié) est-il adapté ici ?

Exercice 20. Fusion de données multi-capteurs

Deux Arduino communiquent par I2C. Chacun envoie ses 10 dernières mesures (triées par timestamp). Le maître doit fusionner les deux flux en un seul tableau trié.

Travail demandé :

1. Implémenter la fusion de deux tableaux triés
2. Contrainte : mémoire limitée, faire la fusion “en place” si possible

```
1 void fusionner(int t1[], int n1, int t2[], int n2, int resultat[]);
```

Extension : Et si on avait 4 capteurs ? Implémenter une fusion k-way.

Exercice 21. Double buffer pour affichage

On contrôle un ruban de 16 LEDs RGB. Pendant qu'on affiche un buffer, on prépare le suivant pour éviter le scintillement.

Travail demandé :

1. Créer deux buffers de 16×3 octets (RGB)
2. Implémenter le swap de buffers (juste échanger les pointeurs, pas les données)
3. Remplir le buffer "back" avec un effet de défilement pendant que le "front" est affiché

```
1 byte buffer1[48], buffer2[48];
2 byte* front = buffer1;
3 byte* back = buffer2;
4
5 void swapBuffers() {
6     byte* temp = front;
7     front = back;
8     back = temp;
9 }
```

Partie V : Matrices de Capteurs**Exercice 22. Détection de points froids**

Une matrice de capteurs de température 4×4 (type AMG8833) détecte les zones froides (point creux = température inférieure à tous les voisins).

Travail demandé :

1. Stocker les 16 mesures dans un tableau 2D
2. Détecter les points creux (minima locaux)
3. Allumer une LED si un point froid est détecté (possible fuite d'air)

```
1 int matrice[4][4];
2 bool estPointCreux(int i, int j);
```

Optimisation : Stocker la matrice en 1D pour un accès plus rapide.

Exercice 23. Balayage de matrice LED

Une matrice LED 8×8 est contrôlée par multiplexage. Pour un affichage fluide, on doit parcourir les lignes en "accordéon" (alternance gauche-droite / droite-gauche) pour optimiser les transitions.

Travail demandé :

1. Stocker l'image en PROGMEM (8 octets)
2. Écrire la routine de balayage avec manipulation directe des ports
3. Appeler cette routine dans une interruption timer


```
1 const byte image[8] PROGMEM = { ... };
2
3 void balayerAccordeon() {
4     for (int i = 0; i < 8; i++) {
5         byte ligne = pgm_read_byte(&image[i]);
6         if (i % 2 == 1) ligne = inverseBits(ligne);
7         PORTD = ligne;
8         // Activer la ligne i...
9     }
10 }
```

Exercice 24. Clavier matriciel – ligne la plus active

Un clavier matriciel 4×4 est scanné régulièrement. On veut déterminer quelle ligne a le plus de touches enfoncées simultanément (détection de “mashing”).

Travail demandé :

1. Scanner le clavier et représenter l'état par 4 octets (4 bits utiles chacun)
2. Compter les bits à 1 dans chaque ligne
3. Retourner l'indice de la ligne avec le maximum

```
1 byte etatClavier[4]; // Bits representent les colonnes pressees
2
3 int lignePlusActive();
```

Exercice 25. Checksum diagonal

Pour vérifier l'intégrité d'une matrice de données 8×8 transmise par radio, on calcule un checksum basé sur les sommes des deux diagonales.

Travail demandé :

1. Calculer la somme de la diagonale principale
2. Calculer la somme de la diagonale secondaire
3. XOR des deux sommes = checksum

```
1 byte calculerChecksum(byte matrice[8][8]);
```

Exercice 26. Compression de matrice binaire

Une matrice 8×8 de booléens (64 bits) doit être stockée en EEPROM de manière compacte.

Travail demandé :

1. Compresser la matrice en 8 octets (1 bit par cellule)
2. Écrire en EEPROM
3. Relire et décompresser

```
1 void compresserMatrice(bool matrice[8][8], byte compresse[8]);
2 void decompresserMatrice(byte compresse[8], bool matrice[8][8]);
```

Partie VI : Problèmes Avancés sous Contraintes Embarquées

Exercice 27. Gestion de périphériques avec tableaux parallèles

On gère jusqu'à 8 capteurs I2C avec :

- Un tableau d'adresses : `byte adresses[8]`
- Un tableau de dernières valeurs : `int valeurs[8]`
- Un tableau de timestamps : `unsigned long timestamps[8]`

Travail demandé :

1. Écrire `lireTousCapteurs()` qui met à jour les tableaux
2. Trouver le capteur avec la valeur maximale
3. Trier les capteurs par valeur (décroissant) en gardant la correspondance adresse-valeur
4. Détecter les capteurs qui n'ont pas répondu depuis plus de 5 secondes

Question : Comment les tableaux parallèles impactent-ils la localité des données et donc les performances ?

Exercice 28. Minimisation de consommation

On a N tâches avec des durées `d[i]` et des priorités `p[i]`. On veut ordonnancer les tâches pour minimiser le temps total pondéré d'attente.

Application Arduino : Ordonnancer les lectures de capteurs pour minimiser la consommation (les capteurs en veille consomment moins).

Travail demandé :

1. Implémenter l'algorithme de tri par ratio priorité/durée
2. Calculer l'ordre optimal de lecture
3. Mesurer l'économie d'énergie (simulation)

Exercice 29. Séquences montantes pour prédiction

On analyse l'historique d'un capteur pour détecter des tendances. Compter les sous-séquences strictement croissantes permet d'évaluer la "nervosité" du signal.

Travail demandé :

1. Implémenter le comptage de sous-séquences croissantes sur un buffer de 50 valeurs
2. Si le nombre dépasse un seuil, déclencher une alarme
3. Optimiser pour ne pas recalculer tout à chaque nouvelle valeur (approche incrémentale)

Exercice 30. Clustering de capteurs

Plusieurs capteurs mesurent la même grandeur physique. On veut regrouper ceux qui donnent des valeurs "proches" (différence $<$ seuil) pour détecter les capteurs défectueux.

Travail demandé :

1. Implémenter Union-Find en mémoire très limitée (tableau de 8 éléments max)

2. Fusionner les capteurs avec des lectures similaires
3. Identifier le plus grand groupe (capteurs “normaux”) et les outliers

```
1 byte parent[8]; // Seulement 8 octets !
2
3 void unionner(byte a, byte b);
4 byte trouverRacine(byte x);
```

Questions Transversales

Question A : Compromis Mémoire/Temps

Pour chaque exercice, réfléchir au compromis entre :

- Complexité temporelle (vitesse d'exécution)
- Complexité spatiale (occupation mémoire)

Sur un PC moderne, on privilégie souvent le temps. Sur Arduino, **la mémoire est souvent le facteur limitant**.

Question B : Volatilité des données

Pour chaque exercice utilisant des tableaux :

1. Les données doivent-elles survivre à un redémarrage ? → EEPROM
2. Sont-elles constantes ? → PROGMEM
3. Sont-elles modifiées par des interruptions ? → **volatile**

Question C : Représentation compacte

Identifier les exercices où une représentation en bits (au lieu d'octets ou d'entiers) permettrait d'économiser significativement de la mémoire.

Bon courage !